The Comedy Cadence - The Math behind the Movie Magic

Exploratory Data Analysis

By - Binni Thomas

Introduction

With special love and passion for the Hollywood cinema (World Cinema) and considering the aspiring film making enthusiasts, this EDA analysis aims at understanding the pace mechanism of comedy featured movies, for over a time period of 50 glorious years (1977-2026). This analysis is driven by a set of objectives, where the importance of using Edits & Dialogues to entertain the audience can be greatly felt. The presence of statistical prediction also makes this analysis more interesting and powerful.

Assumptions

1. Though cinema is a subjective kind of entertainment, a comedy cinema is a matter of enjoyment to all, irrespective of the era or the type of humor.
2. The HOOK, CONFLICT, CLIMAX scenes for all the movies are subjective , hence the duration, number of shots, word counts, and all varies accordingly. (Outlier is not applicable here)

Feature Description

- Movie title - Name of the movie (String)
- Released year – Year of which at the movie was released (Numeric)
- Humor type – Type of the humor seen in a movie (String)
- Scene type – Scenes considered for the study (Hook, Conflict, Climax) (String)
- Duration Sec – Duration of the scene (in seconds) (Numeric)
- Total cuts – Total number of cuts noted in a scene (Numeric)
- Word count – Total number of words within the dialogues in a scene (Numeric)
- Tension score – Tension score for the respective scene types (1-10) (Numeric)
- Notes – Short description of the respective scene types (String)

Objectives

1. To understand the "Visual vs. Verbal" dominance shift through comparison of visual humor and verbal humor.
2. To determine the "Cringe-to-Cut" correlation to ensure a mathematical understanding of using cuts & dialogues mechanism.
3. To frame a "Climax Acceleration" blueprint by understanding the pacing mechanism used within the progress of the movies.
4. To understand and frame the "Era-Specific Rhytm" benchmark, that would benefit the editors to ensure an effective pacing.

```
# Importing the necessary libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
# Loading the data set
hollywood_cinema_data = pd.read_csv('/content/Project_Master_Data.csv')
```

```
# Preliminary view of the data set
hollywood_cinema_data.head(6)
```

	Movie_Title	Released_Year	Humour_Type	Scene_Type	Duration_Sec	Total_Cuts	Word_Count	Tension_Score	Notes
0	Annie Hall	1977	Intellectual Satire	Hook	105	1	304	1	A simple verbal introduction of the main chara...
1	Annie Hall	1977	Intellectual Satire	Conflict	80	2	227	6	Tension is arisen within a funny ambience, whe...

```
# Shape of the data set
hollywood_cinema_data.shape
```

```
(45, 9)
```

```
hollywood_cinema_data.index
```

```
RangeIndex(start=0, stop=45, step=1)
```

```
hollywood_cinema_data.columns
```

```
Index(['Movie_Title ', 'Released_Year', 'Humour_Type', 'Scene_Type',
       'Duration_Sec', 'Total_Cuts', 'Word_Count', 'Tension_Score', 'Notes'],
      dtype='object')
```

Insight: The 'hollywood_cinema_data' comprises of 45 records (15 movies and 3 scenes each), based on 9 relevant features.

```
# Features and data types
hollywood_cinema_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 45 entries, 0 to 44
Data columns (total 9 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   Movie_Title    45 non-null     object
 1   Released_Year  45 non-null     int64
 2   Humour_Type    45 non-null     object
 3   Scene_Type     45 non-null     object
 4   Duration_Sec   45 non-null     int64
 5   Total_Cuts     45 non-null     int64
 6   Word_Count     45 non-null     int64
 7   Tension_Score  45 non-null     int64
 8   Notes          45 non-null     object
dtypes: int64(5), object(4)
memory usage: 3.3+ KB
```

Insight: The data types are seen to be perfect, without any blank cells.

```
# Statistical information
hollywood_cinema_data.describe()
```

	Released_Year	Duration_Sec	Total_Cuts	Word_Count	Tension_Score
count	45.000000	45.000000	45.000000	45.000000	45.000000
mean	2002.466667	294.155556	78.244444	425.733333	6.911111
std	12.960780	195.867529	106.937993	327.126011	2.521023
min	1977.000000	44.000000	1.000000	47.000000	1.000000
25%	1993.000000	188.000000	39.000000	227.000000	5.000000
50%	2007.000000	265.000000	66.000000	352.000000	8.000000
75%	2014.000000	346.000000	85.000000	494.000000	9.000000
max	2020.000000	1055.000000	728.000000	1511.000000	10.000000

```
# Understanding the quantitative part of the data (feature-wise)
hollywood_cinema_data.describe(include="object")
```

	Movie_Title	Humour_Type	Scene_Type	Notes
count	45	45	45	45
unique	15	14	3	45
top	Annie Hall	Cringe Comedy	Hook	A simple verbal introduction of the main chara...
freq	3	6	15	1

Insight: There are 15 unique Comedy special Hollywood movies, each represented under either of the 14 varieties of humour types (Surreal Parody, Stylized Comedy, and so on), for which the data majorly focuses on 3 specific scene types within each movies (Hook, Conflict, Climax). Each scenes are supported with short descriptions (Notes) for better understanding.

Objective 01 : Visual vs. Verbal Dominance

Question - Do modern comedies stop talking and start "cutting" to keep the audience engaged?

Setting up the function & Calculating the metrics

```
def objective1(data,movie_title):
  data.columns = data.columns.str.strip() # To remove the trail spaces in column heads
  movie_data = data[data['Movie_Title'].str.strip() == movie_title] # Filtering for the movie name in the data set

  if movie_data.empty:
    print('No Movies found. Please check the spelling.') # If in case there is a mismatch in the movie title
    return

  released_year = movie_data['Released_Year'].iloc[0] # Retrieving the year of release for a movie

  print(f"Movie - {movie_title.upper()} ({released_year})") # Main header section
  print('*'*65) # Seperation line, for easy readability

  for index, rows in movie_data.iterrows(): # Iteration through the needed features
    scene = rows['Scene_Type']
    duration = rows['Duration_Sec']
    cuts = rows['Total_Cuts']
    words = rows['Word_Count']

    # Metrics 01 - Rhythm Ratio (rr)
    rr = cuts / words if words > 0 else 0 # Just a Provision, if in case the value goes to 0

    # Metrics 02 - Words Per Second (wps)
    wps = words / duration if duration > 0 else 0 # Just a Provision, if in case the value goes to 0

    # Metrics 03 - Cuts Per Minute (cpm)
    cpm = (cuts / duration) * 60 if duration > 0 else 0 # Duration (in Seconds), now converted to Minutes (*60)

    if rr < 0.10 and wps > 1.5: # Benchmarking the metrics for the clear-cut insights
      dominance_style = 'VERBAL DOMINANCE'
      observation = 'Sorkin-style approach - More WORDS, less CUTS'
    elif 0.10 <= rr <= 0.25 and cpm > 10:
      dominance_style = 'BALANCED RHYTHM'
      observation = 'A great synergy between the WORDS and the CUTS'
    elif rr > 0.25 and cpm > 12:
      dominance_style = 'VISUAL DOMINANCE'
      observation = 'Wright-style approach - More CUTS, less WORDS'
    else:
      dominance_style = 'STAGNANT'
      observation = 'A slower-pace between the WORDS and the CUTS'

    print(f"SCENE: {scene.upper()}") # For an organised way of displaying the output
    print(f"Rhythm Ratio: {rr:.3f} (Cuts per Word)")
    print(f"Delivery Pace: {wps:.2f} W/s (Words per Second)")
    print(f"Edit Velocity: {cpm:.2f} CPM (Cuts per Minute)")
    print(f"VERDICT: {dominance_style}")
    print(f"INSIGHT: {observation}")
    print("-"*65) # A seperation line for easy readability
```

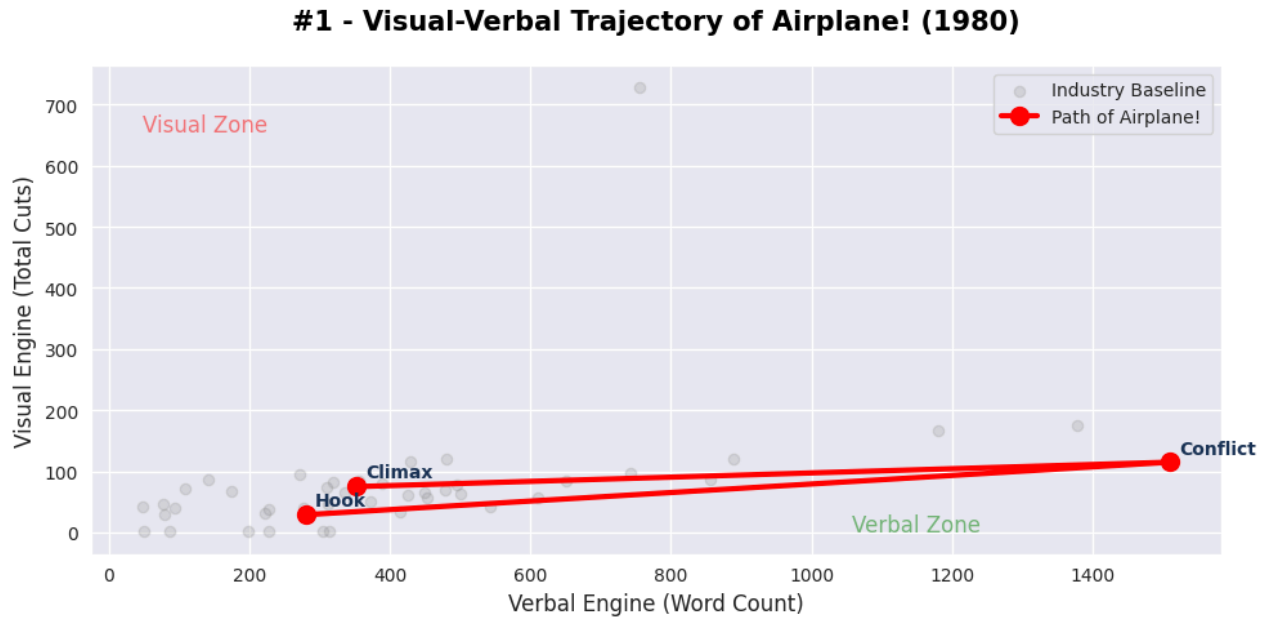
```
objective1(hollywood_cinema_data,'Airplane!')
```

```
Movie - AIRPLANE! (1980)
*****************************************************************
SCENE: HOOK
Rhythm Ratio: 0.104 (Cuts per Word)
Delivery Pace: 1.49 W/s (Words per Second)
Edit Velocity: 9.26 CPM (Cuts per Minute)
VERDICT: STAGNANT
INSIGHT: A slower-pace between the WORDS and the CUTS
-----------------------------------------------------------------
SCENE: CONFLICT
Rhythm Ratio: 0.076 (Cuts per Word)
Delivery Pace: 1.84 W/s (Words per Second)
Edit Velocity: 8.40 CPM (Cuts per Minute)
VERDICT: VERBAL DOMINANCE
INSIGHT: Sorkin-style approach - More WORDS, less CUTS
-----------------------------------------------------------------
SCENE: CLIMAX
Rhythm Ratio: 0.213 (Cuts per Word)
Delivery Pace: 1.28 W/s (Words per Second)
Edit Velocity: 16.42 CPM (Cuts per Minute)
VERDICT: BALANCED RHYTHM
INSIGHT: A great synergy between the WORDS and the CUTS
-----------------------------------------------------------------
```

Plotting a Cinematic Trajectory Map (Scene-by-scene pathway)

```
def cinema_trajectory_map(data, movie_title):
  movie_name = movie_title.strip()

  target_movie = data[data['Movie_Title'].str.strip() == movie_name].copy() # Considering only the movie under the limeligh
  released_year = target_movie['Released_Year'].iloc[0]
  others = data[data['Movie_Title'].str.strip() != movie_name] # Considering other movies

  scene_order = {'Hook': 0, 'Conflict': 1, 'Climax': 2} # Target movie segregated by scene-order
  target_movie['order'] = target_movie['Scene_Type'].map(scene_order)
  target_movie = target_movie.sort_values('order')

  # Creating the plot
  plt.figure(figsize=(10,5))
  sns.set_style("darkgrid")


  plt.scatter(others['Word_Count'], others['Total_Cuts'], # Specifications for other movies
              alpha=0.2, c='gray', label='Industry Baseline')

  plt.plot(target_movie['Word_Count'], target_movie['Total_Cuts'], # Specifications for the target movie
           marker='o', markersize=10, linewidth=3, color='Red',
           label=f'Path of {movie_name}')

  for i, row in target_movie.iterrows(): # Scene-wise information marking
        plt.annotate(f"{row['Scene_Type']}", (row['Word_Count'], row['Total_Cuts']),
                     textcoords="offset points", xytext=(5,5), fontweight='bold', fontsize=10, color='#1d3557') # Setting u

  plt.title(f"#1 - Visual-Verbal Trajectory of {movie_name} ({released_year})", color='black', fontsize=15, fontweight='bol
  plt.xlabel("Verbal Engine (Word Count)", fontsize=12)
  plt.ylabel("Visual Engine (Total Cuts)", fontsize=12)
  plt.legend()

  plt.text(data['Word_Count'].max()*0.7, data['Total_Cuts'].min(), "Verbal Zone", alpha=0.5, fontsize=12, color='green') #
  plt.text(data['Word_Count'].min(), data['Total_Cuts'].max()*0.9, "Visual Zone", alpha=0.5, fontsize=12, color='red')
  plt.tight_layout()

  plt.show()
```

```
cinema_trajectory_map(hollywood_cinema_data,'Airplane!')
```



Insight:

On a sample note, considering one movie, prior to the Millenium (2000).

Airplane! breaks standard industry patterns by using an extreme amount of dialogue to drive its conflict. While most movies stay in a balanced "Industry Baseline" zone, Airplane! swings deep into the "Verbal Zone" before shifting back to a visual focus for its finish. This shows the film relies on a unique, talk-heavy rhythm to build tension rather than just using visual action.

Objective 02 : Cringe-to-Cut Correlation

Question - Does "Awkwardness" mathematically require the camera to stay still (long shots) while the tension rises?

Setting up the function & Calculating the metrics

```
def objective2(data, movie_title):
  movie_name = movie_title.strip()
  movie_data = data[data['Movie_Title'].str.strip() == movie_name]

  if movie_data.empty:
    print(f"No Movies found. Please check the spelling.")
    return

  released_year = movie_data['Released_Year'].iloc[0]

  print(f"Movie - {movie_name.upper()} ({released_year})")
  print('*'*85)

  for index, row in movie_data.iterrows():
    scene = row['Scene_Type']
    duration = row['Duration_Sec']
    cuts = row['Total_Cuts']
    tension = row['Tension_Score']

    # Metric 01 - Average Shot Length (asl)
    asl = duration / cuts if cuts > 0 else duration

    # Metric 02 - Cuts Per Minute (cpm)
    cpm = (cuts / duration) * 60 if duration > 0 else 0

    # Benchmarking the metrics
    if asl >= 6.0 and tension >= 6:
      verdict = "CRINGE TENSION"
      observation = "The director holds the shot intentionally long to make the audience feel the awkwardness or dread."
    elif asl < 4.0 and tension >= 6:
      verdict = "PANIC TENSION"
      Observation  = "High tension is conveyed through rapid, chaotic editing rather than lingering discomfort."
    elif asl >= 6.0 and tension < 6:
      verdict = "ATMOSPHERIC SETUP"
      observation = "Low tension with long shots. The scene is likely building world-context or casual character developmer
    else:
      verdict = "STANDARD PACING"
      observation = "Neutral comedic pacing. Neither heavily rushed nor intentionally awkward."

    print(f"SCENE: {scene.upper()}")
    print(f"Tension Score: {tension}/10")
    print(f"Average Shot Length: {asl:.2f} Seconds")
    print(f"Edit Velocity: {cpm:.2f} Cuts per Minute")
    print(f"VERDICT: {verdict}")
    print(f"INSIGHT: {observation}")
    print("-"*85)
```

```
objective2(hollywood_cinema_data,'Mean Girls')
```

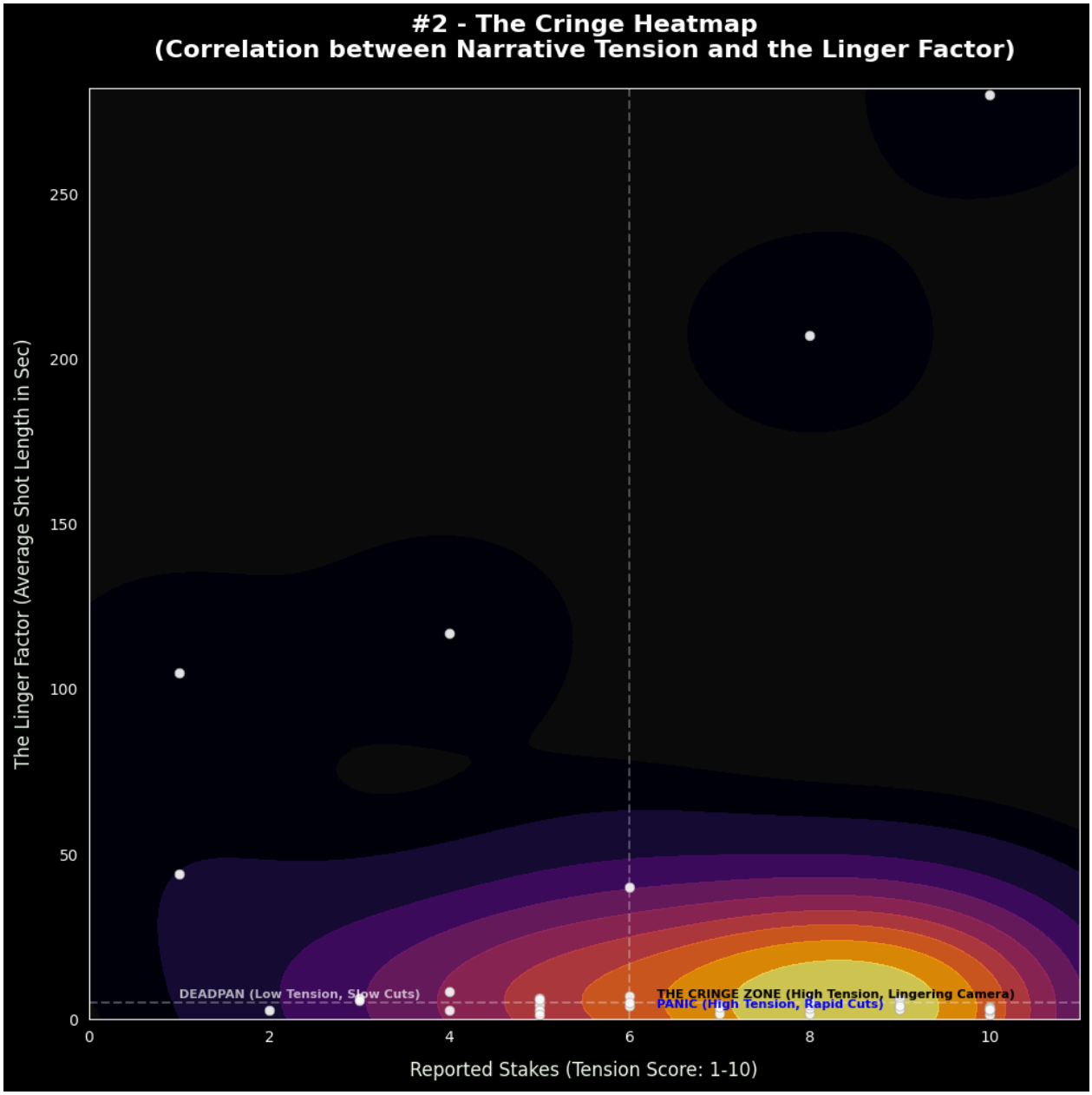
```
Movie - MEAN GIRLS (2004)
*************************************************************************************
SCENE: HOOK
Tension Score: 4/10
Average Shot Length: 2.65 Seconds
Edit Velocity: 22.60 Cuts per Minute
VERDICT: STANDARD PACING
INSIGHT: Neutral comedic pacing. Neither heavily rushed nor intentionally awkward.
-------------------------------------------------------------------------------------
SCENE: CONFLICT
Tension Score: 8/10
Average Shot Length: 4.34 Seconds
Edit Velocity: 13.82 Cuts per Minute
VERDICT: STANDARD PACING
INSIGHT: Neutral comedic pacing. Neither heavily rushed nor intentionally awkward.
-------------------------------------------------------------------------------------
SCENE: CLIMAX
Tension Score: 9/10
Average Shot Length: 3.43 Seconds
Edit Velocity: 17.48 Cuts per Minute
VERDICT: PANIC TENSION
INSIGHT: Neutral comedic pacing. Neither heavily rushed nor intentionally awkward.
-------------------------------------------------------------------------------------
```

Plotting a Cringe Heatmap (Cringe zone)

```
def cringe_heatmap(data):

  data['asl'] = data['Duration_Sec'] / data['Total_Cuts']
```

```
plt.style.use('dark_background')
fig, ax = plt.subplots(figsize=(10,10))
ax.set_facecolor('#0a0a0a') # This code denotes a deep theatrical black color

# Setting up the heatmap architecture
sns.kdeplot(
      x=data['Tension_Score'],
      y=data['asl'],
      cmap="inferno",
      fill=True,
      thresh=0.05,
      bw_adjust=0.85,
      ax=ax,
      alpha=0.85)

# Plotting the scatterplot
sns.scatterplot(
      x=data['Tension_Score'],
      y=data['asl'],
      color='white',
      s=40,
      edgecolor='#aaaaaa',
      alpha=0.9,
      ax=ax,
      zorder=5)

tension_threshold = 6.0 # Threshold lines
asl_threshold = 5.0
plt.axvline(x=tension_threshold, color='white', linestyle='--', alpha=0.3, zorder=4)
plt.axhline(y=asl_threshold, color='white', linestyle='--', alpha=0.3, zorder=4)

# Zone labelling for easy identification
plt.text(tension_threshold + 0.3, asl_threshold + 1.5, "THE CRINGE ZONE (High Tension, Lingering Camera)",
           color='Black', fontsize=8, fontweight='bold', zorder=6)

plt.text(tension_threshold + 0.3, asl_threshold - 1.5, "PANIC (High Tension, Rapid Cuts)",
           color='Blue', fontsize=8, fontweight='bold', zorder=6)

plt.text(1, asl_threshold + 1.5, "DEADPAN (Low Tension, Slow Cuts)",
           color='White', fontsize=8, fontweight='bold', alpha=0.7, zorder=6)

plt.title("#2 - The Cringe Heatmap\n(Correlation between Narrative Tension and the Linger Factor)",
            fontsize=16, fontweight='bold', color='white', pad=20)
plt.xlabel("Reported Stakes (Tension Score: 1-10)", fontsize=12, labelpad=10, color='#f1faee')
plt.ylabel("The Linger Factor (Average Shot Length in Sec)", fontsize=12, labelpad=10, color='#f1faee')

# Boundary setting for a balanced graph display
plt.xlim(0, 11)
plt.ylim(0, data['asl'].max() + 2)

plt.grid(False)
plt.tight_layout()

plt.show()
```

```
cringe_heatmap(hollywood_cinema_data)
```



Insight:

Real "cringe" happens when a camera refuses to look away during a tense moment. While most high-stakes scenes use fast "Panic" cuts, the outliers at the top of the chart hold a single shot for over 250 seconds. By lingering this long, the filmmaker forces you to sit through the awkwardness without giving you a visual break.

Objective 03 : The Climax Acceleration Blueprint

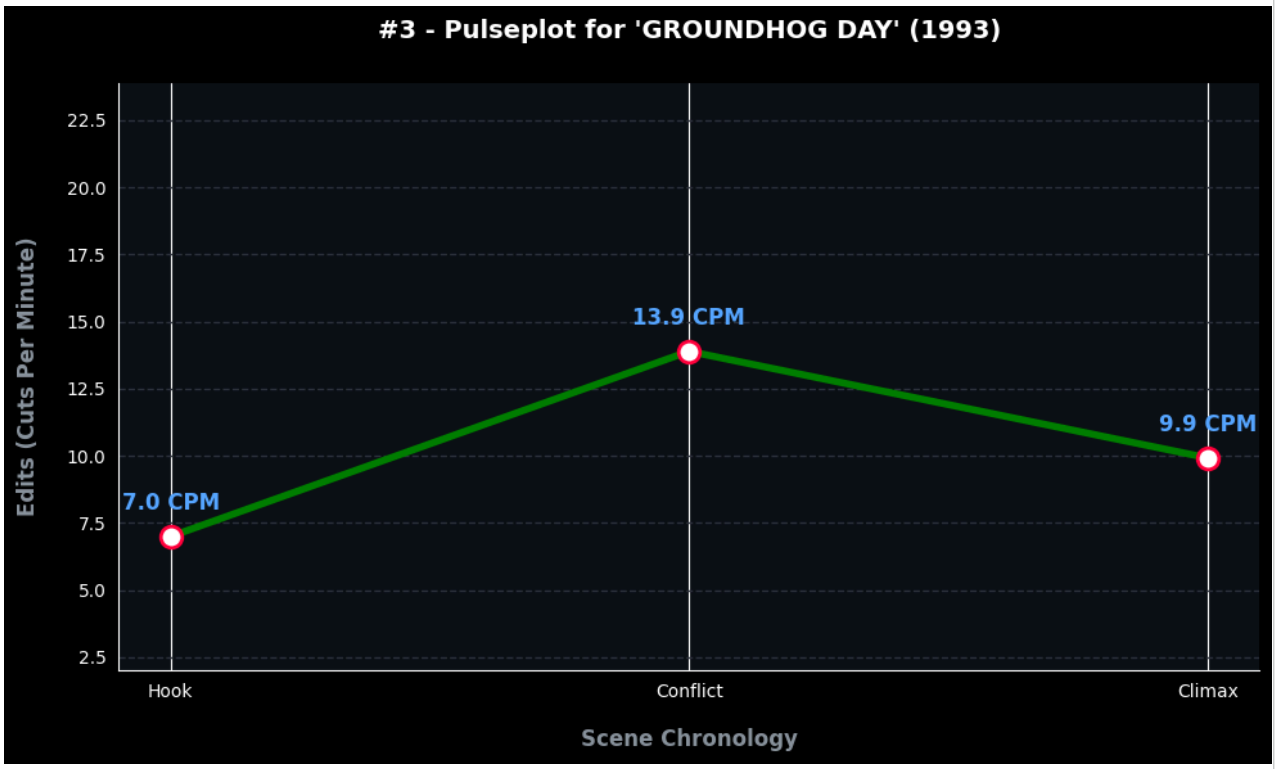
Question - Does the editing frequency (CPM) always accelerate as we move from Hook to Climax?

Setting up the function to plot a Pulse graph

```
def pulseplot(df, movie_title):

    movie_name = movie_title.strip() # Information considered from the main data (hollywood_cinema_data)
    movie_data = df[df['Movie_Title'].str.strip() == movie_name].copy()
    released_year = movie_data['Released_Year'].iloc[0]

    # Metric considered - Cuts per Minute (cpm)
    movie_data['cpm'] = (movie_data['Total_Cuts'] / movie_data['Duration_Sec']) * 60

    # Mapping the scene order as per the chronology (Hook - Conflict - Climax)
    scene_order = {'Hook': 1, 'Conflict': 2, 'Climax': 3}
    movie_data['story_order'] = movie_data['Scene_Type'].str.strip().map(scene_order)
```

```
    movie_data = movie_data.sort_values('story_order')

    plt.style.use('dark_background') # Plotting phase - Background setup
    fig, ax = plt.subplots(figsize=(10, 6))
    ax.set_facecolor('#0d1117') # Deep studio color

    # Pulseplot - Aesthetics
    ax.plot(
        movie_data['Scene_Type'],
        movie_data['cpm'],
        color='Green',
        marker='o',
        markersize=12,
        linewidth=4,
        markerfacecolor='white',
        markeredgecolor='#ff003c',
        markeredgewidth=2,
        zorder=3)

    for index, row in movie_data.iterrows():
      ax.annotate(
          f"{row['cpm']:.1f} CPM",
            (row['Scene_Type'], row['cpm']),
            textcoords="offset points",
            xytext=(0, 15),
            ha='center',
            fontsize=12,
            fontweight='bold',
            color='#58a6ff')

    plt.title(f"#3 - Pulseplot for '{movie_name.upper()}' ({released_year})",
              fontsize=14, fontweight='bold', color='white', pad=25)
    plt.xlabel("Scene Chronology", fontsize=12, labelpad=15, color='#8b949e', fontweight='bold')
    plt.ylabel("Edits (Cuts Per Minute)", fontsize=12, labelpad=15, color='#8b949e', fontweight='bold')

    y_min, y_max = movie_data['cpm'].min(), movie_data['cpm'].max() # Y-Axis threshold
    plt.ylim(max(0, y_min - 5), y_max + 10)

    ax.grid(axis='y', color='#30363d', linestyle='--', linewidth=1, zorder=1) # Seperation grid
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)

    plt.tight_layout()
    plt.show()
```

```
pulseplot(hollywood_cinema_data,'Groundhog Day')
```



Insight:

On a sample note, considering one movie, prior to the Millenium (2000).

The movie is most visually frantic in the middle rather than at the end. Editing speed nearly doubles from the "Hook" (7.0 CPM) to reach a peak of 13.9 cuts per minute during the "Conflict". Because the pace slows down to 9.9 cuts for the "Climax," the graph shows that the peak intensity happens during the struggle rather than the finale.

Objective 04 : Era-Specific Rhythm

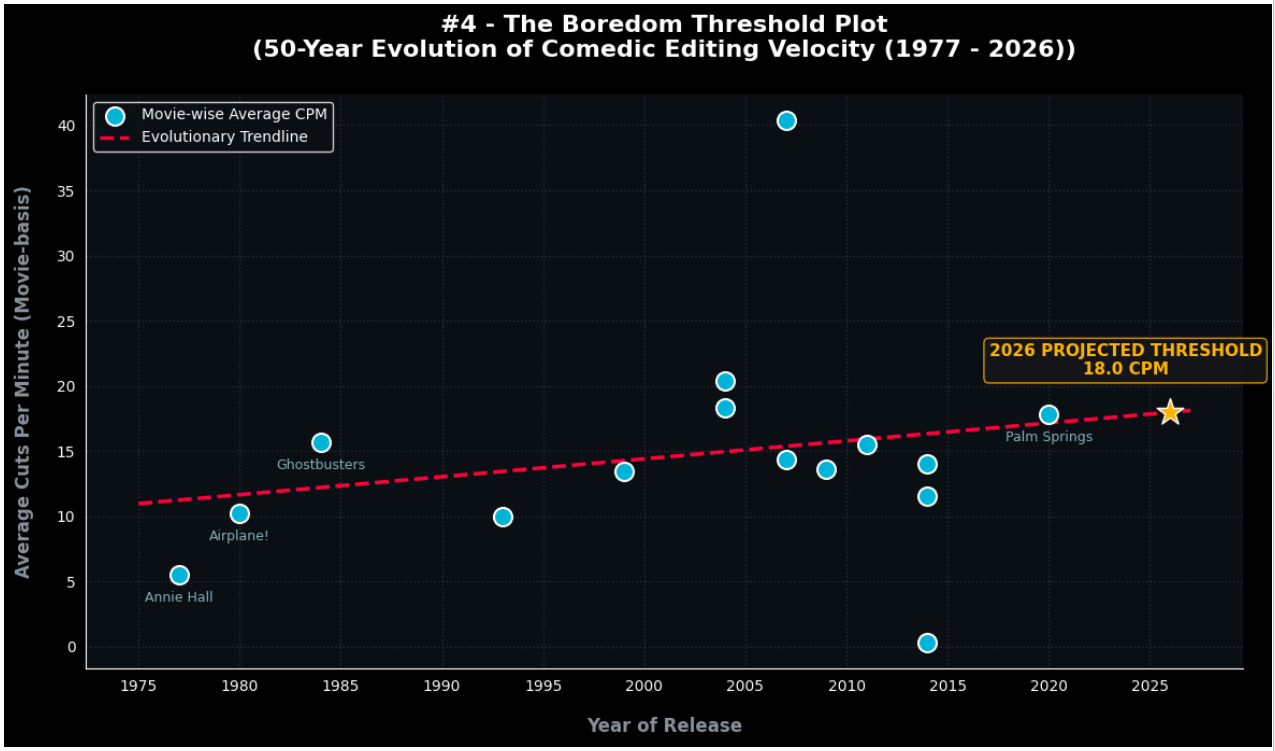
Question - Has the "Standard" for a comedy cut changed since 1977?

Setting up the function to plot a graph

```
def era_boredom_threshold(df):

  # Aggregating Edits and Duration for each movies
  movie_era_data = df.groupby(['Movie_Title', 'Released_Year']).agg({'Total_Cuts': 'sum','Duration_Sec': 'sum'}).reset_index

  # Metric considered - Movie-wise Cuts Per Minute (mcpm)
  movie_era_data['mcpm'] = (movie_era_data['Total_Cuts'] / movie_era_data['Duration_Sec']) * 60
  movie_era_data = movie_era_data.sort_values('Released_Year')

  plt.style.use('dark_background') # Setting up the background
  fig, ax = plt.subplots(figsize=(12, 7))
  ax.set_facecolor('#0d1117')

  # Considering the year-wise historical CPM for plotting
  sns.scatterplot(
      data=movie_era_data,
      x='Released_Year',
      y='mcpm',
      s=150,
      color='#00b4d8',
      edgecolor='white',
      linewidth=1.5,
      ax=ax,
      zorder=3,
      label='Movie-wise Average CPM')

  x = movie_era_data['Released_Year'] # Statistical trendline
  y = movie_era_data['mcpm']

  z = np.polyfit(x, y, 1) # Line of best fit
  p = np.poly1d(z)

  future_years = np.arange(x.min() - 2, 2028) # Timeline extension (Until the year 2026)

  # Plotting the trendline
  ax.plot(
      future_years,
      p(future_years),
      color='#ff003c',
      linestyle='--',
      linewidth=2.5,
      zorder=2,
      label='Evolutionary Trendline')

  # Decorating the 2026 prediction part
  forecast_2026 = p(2026)
  ax.scatter(2026, forecast_2026, color='#ffb703', s=350, marker='*', edgecolor='white', zorder=4)

  ax.annotate(
      f"2026 PROJECTED THRESHOLD\n{forecast_2026:.1f} CPM",
      (2026, forecast_2026),
      textcoords="offset points",
      xytext=(-30, 25),
      ha='center',
      fontsize=11,
      fontweight='bold',
      color='#ffb703',
      bbox=dict(boxstyle="round,pad=0.3", fc="#1a1a1a", ec="#ffb703", alpha=0.8))

  # Annotating Old vs New movies for easy identification
  for index, row in movie_era_data.iterrows():
    if row['Released_Year'] < 1985 or row['Released_Year'] > 2018:
      ax.annotate(
          row['Movie_Title'].strip(),
              (row['Released_Year'], row['mcpm']),
              textcoords="offset points",
              xytext=(0, -18),
```

```
                ha='center',
                fontsize=9,
                color='#a8dadc',
                alpha=0.8)

    # Main plot
    plt.title("#4 - The Boredom Threshold Plot\n(50-Year Evolution of Comedic Editing Velocity (1977 - 2026))",
                fontsize=16, fontweight='bold', color='white', pad=25)
    plt.xlabel("Year of Release", fontsize=12, labelpad=15, color='#8b949e', fontweight='bold')
    plt.ylabel("Average Cuts Per Minute (Movie-basis)", fontsize=12, labelpad=15, color='#8b949e', fontweight='bold')

    ax.grid(axis='both', color='#30363d', linestyle=':', linewidth=1, zorder=1) # Additional features
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)
    plt.xticks(np.arange(1975, 2030, 5))
    ax.legend(loc='upper left', facecolor='#0d1117', edgecolor='white', fontsize=10)

    plt.tight_layout()
    plt.show()
```

```
era_boredom_threshold(hollywood_cinema_data)
```



Insight:

Comedy is becoming visually faster and more "manic" over time. While older classics like Annie Hall (1977) and Airplane! (1980) had a relatively relaxed pace of 5 to 10 cuts per minute (CPM), modern comedies are trending toward a projected threshold of 18.0 CPM by 2026. This evolution suggests that filmmakers are increasingly relying on rapid editing to keep modern audiences engaged, moving away from the slower, dialogue-driven pacing of the past.

Conclusion

Comedy movies are always special to the audience. In the light of this analysis, an era specific viewpoint states that the old comedies (Pre 2000s) were majorly driven verbally (More of dialogues than the edits). But post 2000s, the comedy mechanism started to be driven through the use of edits, even more rigorous and beyond. Hence, the skill of Directors and Editors are more at stake, not that, scriptwriters are to be neglected, but since the taste of audience change as the time progresses, this is a truth to be taken into the limelight.